

Continuous deformation based panelization for design rationalization

Elias Jadon
elias.jadon1@gmail.com
University of Haifa
Israel

Bernhard Thomaszewski
Aleksandra Anna Apolinarska
bthomasz@ethz.ch
apolinarska@arch.ethz.ch
ETH Zürich
Switzerland

Roi Poranne
roi.poranne@gmail.com
University of Haifa
Israel

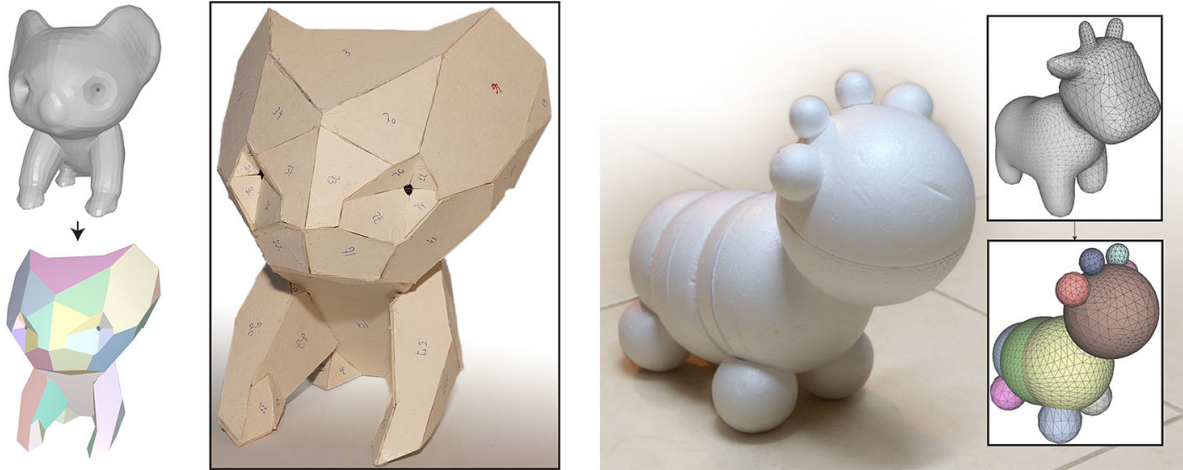


Figure 1: Our method can deform a shape and create planar parts that can be manufactured, e.g., from cardboard. Our method can also take a mesh and deform it into a shape made entirely of spherical parts. The parts can be cut from styrofoam balls and assembled.

ABSTRACT

Design rationalization is the process of simplifying a 3D shape to enable cost-efficient manufacturing. A common approach is to approximate the input shape by a collection of simple units, such as flat or spherical panels, that are easy to manufacture and simple to assemble. This *panelization* process typically involves a segmentation step, with each surface patch intended to be replaced by a single unit, followed by an approximation stage, where the final shapes and locations of the units are determined. While optimal panel parameters for given segments are readily determined, the discrete nature of segmentation—assigning surface elements to segments—prevents a continuous design optimization workflow. In this work, we propose a differentiable reformulation of panelization that enables its use in gradient-based design optimization. Our approach is to treat panelization as a smooth optimization problem, whose

objective function encourages the surface to locally deform towards best-matching units. This formulation enables a fully-differentiable rationalization process with implicit segmentation in which panels emerge automatically. We integrate our rationalization process in a simple user interface allowing the designer to guide the optimization towards desired panelizations. We demonstrate the potential of our approach on a diverse set of complex shapes and different panel types.

CCS CONCEPTS

• Theory of computation → Nonconvex optimization; • Computing methodologies → Mesh geometry models.

KEYWORDS

computational fabrication, geometry processing

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
SA '22 Conference Papers, December 6–9, 2022, Daegu, Republic of Korea

© 2022 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-9470-3/22/12...\$15.00
<https://doi.org/10.1145/3550469.3555414>

ACM Reference Format:

Elias Jadon, Bernhard Thomaszewski, Aleksandra Anna Apolinarska, and Roi Poranne. 2022. Continuous deformation based panelization for design rationalization. In *SIGGRAPH Asia 2022 Conference Papers (SA '22 Conference Papers)*, December 6–9, 2022, Daegu, Republic of Korea. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3550469.3555414>

1 INTRODUCTION

Recent years saw a surge in research on computational fabrication [Bermano et al. 2017; Bickel et al. 2018], including many applications in architectural and industrial design. In this context, the term *rationalization* refers to the process of simplifying a shape envisioned by the designer in order to make fabrication cost-efficient or even just feasible [Austern et al. 2018]. One common approach is *panelization*, e.g. the approximation of a shape by a collection of basic panels that can be manufactured efficiently. Planar panels are most often used thanks to their simplicity, but other forms, such as spherical or cylindrical panels can also be easily produced. However, automatic methods that can compute such approximations are scarce.

In general, architectural design is a long and tedious process that consists of many different steps, starting from an original design, through rationalization and other steps such as form-finding, fabrication, assembly, etc. Each step introduces more real-world limitations, which impact the design and further separates it from its conception. In particular, a rationalized design often differs substantially from the designer’s vision, and these deviations are then further exacerbated. Thus, we argue, designers would greatly benefit by using an interactive rationalization tool, that could provide predictable results early in the design process.

Toward this end, our goal in this paper is to formulate panelization as a smooth optimization problem, for a variety of panel shapes. This formulation enables a panelization user-interface that reacts to user inputs immediately, and that can potentially be integrated into other optimization tasks. The challenge in reaching a continuous formulation stems from the somewhat discrete nature of the problem, as previous papers approached this problem via clustering. Our approach is to formulate panelization through *continuous deformation*, with a *local* deformation energy that encourages the formation of panels. Our formulation presents multiple benefits:

- (1) It behaves predictably w.r.t. small changes in parameters.
- (2) Computations are local and easily parallelizable.
- (3) Different panel shapes can be treated.

We demonstrate our approach for panelization with planar, spherical and cylindrical panels, and discuss the general construction for other panel shapes. Fabricated results can be seen in Fig. 1. The user interface and further results are shown in the accompanying video.

2 RELATED WORK

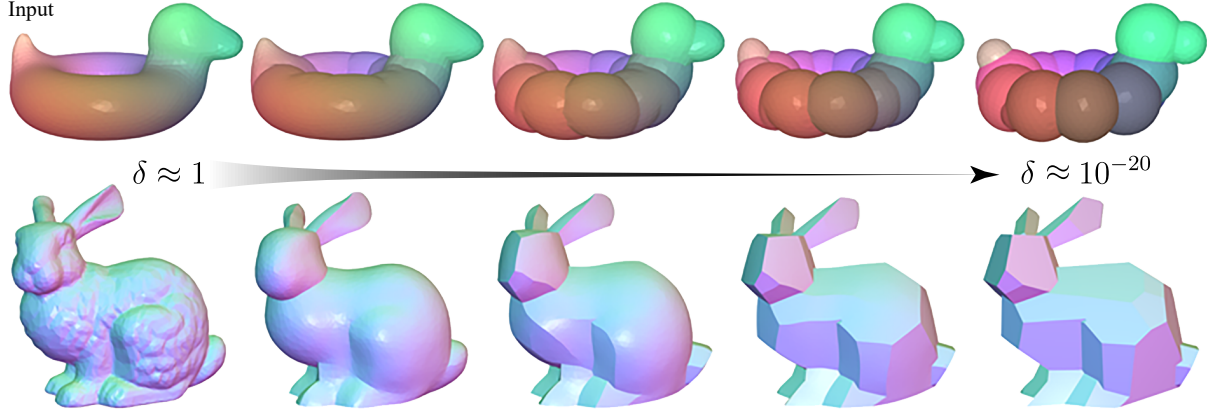
Rationalization. A core problem in architectural geometry, and an instance of rationalization, is to approximate a smooth surface by a polyhedral mesh. Here, a polyhedral mesh is a surface mesh with mostly nontriangular (e.g., quadrilateral or hexagonal) but *planar* faces. Previous solutions to this problem either assume the polyhedral mesh topology is specified by the user or determine it with an initial tessellation stage that segments the smooth target surface into a mesh where each face is *almost* flat [Pottmann et al. 2015]. These papers generally attempt to find highly regular surface tessellations, using specialized quad meshing [Liu et al. 2006] or hex meshing techniques [Li et al. 2015; Pluta et al. 2021]. As such, they tend to be limited in the geometries they are able to handle, in comparison to methods that produce unstructured meshes. Meshing

is followed by a process called *planarization* [Bouaziz et al. 2012; Cutler and Whiting 2007; Tang et al. 2014; Zimmer et al. 2012b], in which the mesh is further optimized to flatten the faces completely.

Panelization, which should not be confused with planarization, is the segmentation of the surface and generation of distinct panels that approximate the geometry of the surface. If all panels can be cut from identical components, then there exists a potential for these components to be mass produced, thus reducing the costs. For this reason, [Eigensatz et al. 2010] proposed an approach to approximate faces by spherical, cylindrical or toroidal panels. This approach is similar to planarization in the sense that it requires a good initial tessellation in order to generate reasonable results. Other proposed to approximate the shape using few identical triangle shapes [Singh and Schaefer 2010; Zimmer et al. 2012a]. Yet another approach is to limit the shape space to surfaces that can easily be panelized using identical panels [Pellis et al. 2021]. Other forms of rationalization employ ruled surfaces (e.g. [Flöry and Pottmann 2010; Wang and Elber 2014]) or developable surfaces (e.g. [Ion et al. 2020; Sellán et al. 2020]). Our approach is a quite different as it deforms the mesh to generate panels. The exception is [Stein et al. 2018] that deforms a mesh to obtain developable patches.

Fabrication. Many works proposed various fabrication techniques that use basic building blocks such as panels. These take an arbitrary shape and create a best-fit approximation for the specific fabrication method [Hildebrand et al. 2012; Holroyd et al. 2011; McCrae et al. 2011]. Similar to our approach, Chen et al. [2013] deform a 3D mesh such that its faces align to form together a small number of planar polygons. Their approach first labels each face and assigns it to one of a predefined number of polygons by solving a discrete optimization problem. It then proceeds to solve another optimization problem that performs the deformation, such that the mesh remains closed and free of self-intersections. In contrast, our method does not require labeling since panels emerge through minimizing a deformation energy.

Shape approximation. Approximating arbitrary shapes by polyhedral meshes is a problem that has been explored previously. Cohen-Steiner et al. [2004] propose a *clustering*-based approach that first groups faces together based on normal deviation and then fits a plane to each cluster, resulting in an unstructured polyhedral mesh. Chen et al. [2013] present a similar but more robust approach for the purpose of fabrication. In contrast to these clustering methods, our approach is based on continuous optimization, and allows the user to smoothly transition between the original surface and its panelized approximation. Our approach for planar panels shares some similarities with [Liu and Jacobson 2019, 2021], which uses the shape normals as an artistic way to make shape analogies. These methods, however, cannot create well defined panels of arbitrary shapes, and rely on preset normal directions to perform. Finally, we mention that our approach is based on L_0 optimization. L_0 optimization has been employed in geometry processing before, e.g. for spline approximation [Brandt et al. 2015] and in a more similar context, for mesh denoising [He and Schaefer 2013]. While the latter allows sharp creases to emerge, it does not generally produce flat panels, or can be generalized to other panel shapes.

Figure 2: Progression of the panelization process as δ is reduced.

3 METHOD

3.1 Problem statement

Given an input triangle mesh our goal is to approximate it by a collection of panels. These panels can be of any shape that can be described parametrically, but we focus on planar and spherical panels in this paper. The approach we propose is to gradually deform the mesh in a way that encourages adjacent faces to belong to the same panel. Taking planar panels as an example, we can achieve this goal by minimizing the *difference* between the normals of adjacent faces in the ℓ_0 sense.

Why ℓ_0 ? To belong to the same panels, the faces must have *exactly* the same normal. Conversely, differences between normals of adjacent panels should not be penalized. Therefore, the penalty term should rapidly increase from zero to an essentially constant maximum value as the normals begin to deviate. This reasoning suggests that the differences should be measured in the ℓ_0 sense. We note that similar considerations were suggested in [Poranne et al. 2017], and we follow a similar strategy later.

Problem formulation. We assume that panels are described by a set of parameters, e.g., an offset from the origin and a normal for the planar case. We endow each input mesh triangle i with its own set of panel parameters, $\mathbf{p}_i$, and denote its vertex positions by $\mathbf{x}_i$. We can represent the requirement that $\mathbf{x}_i$ lie on the panel defined by $\mathbf{p}_i$ using constraints $C(\mathbf{x}_i, \mathbf{p}_i) = 0$ as detailed in 3.2. We stack all positions and parameters in vectors $\mathbf{x}$ and $\mathbf{p}$. For adjacent triangles i and j that share an edge, we define $D(\mathbf{p}_i, \mathbf{p}_j) = \|\mathbf{p}_i - \mathbf{p}_j\|$ to be a distance measure between the two parameter vectors $\mathbf{p}_i$ and $\mathbf{p}_j$, and collect all distances in a single vector $\mathbf{d}(\mathbf{p}) = (D(\mathbf{p}_i, \mathbf{p}_j))_{ij}$. We chose to use the regular ℓ_2 norm for simplicity, although other distance measures can also be used. We pose the following constrained optimization for the full set of parameters $\mathbf{p}$ and vertex positions $\mathbf{x}$ that encourages identical base units for adjacent triangles,

$$\begin{aligned} \min_{\mathbf{x}, \mathbf{p}} \quad & \|\mathbf{d}(\mathbf{p})\|_0 + w_R R(\mathbf{x}) \\ \text{s.t.} \quad & C_h(\mathbf{x}_i, \mathbf{p}_i) = 0 \quad \forall i, \end{aligned} \quad (1)$$

where $\|\cdot\|_0$ is the ℓ_0 norm, that is, the number of non-zeros in the vector, and $R(\mathbf{x})$ contains additional objectives, which we detail in Sec. 3.3. Following [Poranne et al. 2017], we replace the discontinuous ℓ_0 norm with a smoothed version $\|\cdot\|_\delta$ defined by a family of smooth sigmoid functions that can be sharpened gradually. We use

$$\|t\|_\delta = \sum \frac{t_i^2}{t_i^2 + \delta}$$

which is shown in the inset for different values of δ . The progression of the optimization for decreasing values of delta is shown in Fig. 2 and the accompanying video. In practice, we use soft constraints and replace $C_h(\mathbf{x}_i, \mathbf{p}_i) = 0$, by a panel *shape objective* $C(\mathbf{x}_i, \mathbf{p}_i)$ which makes (1) take the form

$$\min_{\mathbf{x}, \mathbf{p}} \quad \|\mathbf{d}(\mathbf{p})\|_0 + w_c \sum C(\mathbf{x}_i, \mathbf{p}_i) + w_R R(\mathbf{x}). \quad (2)$$

3.2 Panel shape objectives

The objective $C(\mathbf{x}_i, \mathbf{p}_i)$ defines the desired shape of the panel. Planar panels are fully described using a normal and an offset, but for our purposes it is enough to endow each face i with a normal $\mathbf{n}_i$ alone. The panel constraint would simply be $\mathbf{n}_i^T \mathbf{E}_i = 0$ where $\mathbf{n}_i$ is treated as a column vector and $\mathbf{E}_i = \mathbf{E}(\mathbf{x}_i)$ is a matrix whose columns are the face's edge vectors. In addition, to avoid a trivial solution, we require that $\|\mathbf{n}_i\| = 1$. The panel shape objective thus is

$$C_{\text{plane}}(\mathbf{x}_i, \mathbf{n}_i) = \|\mathbf{n}_i^T \mathbf{E}_i\|^2 + (\|\mathbf{n}_i\|^2 - 1)^2, \quad (3)$$

and the optimization problem for planar panelization becomes

$$\min_{\mathbf{x}, \mathbf{n}} \quad \|\mathbf{n}_i - \mathbf{n}_j\|_\delta + w_c \sum \|\mathbf{n}_i^T \mathbf{E}_i\|^2 + (\|\mathbf{n}_i\|^2 - 1)^2 + w_R R(\mathbf{x}). \quad (4)$$

In a similar manner, to obtain spherical panels, we add to each face i a sphere center $\mathbf{c}_i$ and radius r_i as auxiliary variables. We then require each vertex of face i to lie on the sphere defined by $\mathbf{c}_i$

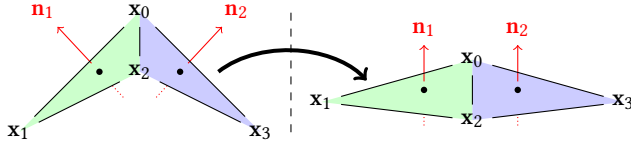


Figure 3: An illustration showing two triangles and their normals before and after matching.

and r_i . The shape objective in this case takes the form

$$C_{\text{sphere}}(\mathbf{x}_i, \mathbf{c}_i, r_i) = \sum_{k=1}^3 (\|\mathbf{x}_i^k - \mathbf{c}_i\|^2 - r_i^2)^2, \quad (5)$$

where $\mathbf{x}_i^k$ is the k -th vertex of i .

Finally, for cylindrical panels we introduce the parameters $\mathbf{v}_i$ and $\mathbf{d}_i$ which represent the direction and offset of the generator of each cylinder. Then the cylindrical shape objective is

$$C_{\text{cyl}}(\mathbf{x}_i, \mathbf{v}_i, \mathbf{d}_i) = \sum_{k=1}^3 (\|\mathbf{v}_i \times (\mathbf{x}_i^k - \mathbf{d}_i)\|^2 - r_i^2)^2, \quad (6)$$

An example is shown in Fig. 5 and the accompanying video.

In practice, mass-produced spherical or cylindrical components are only available in a discrete set of radii. To force the panels to take on specific radii, we use an additional objective—a piecewise quadratic function—with minima at the corresponding diameter values. For example, to encourage integer size from 1 to 10 we use,

$$C_{\text{rad}}(r) = \begin{cases} (r-1)^2, & r < 1 \\ (r - \text{round}(r))^2, & 1 \leq r \leq 10 \\ (r-10)^2, & r > 10 \end{cases} \quad (7)$$

as shown in the inset. We note that the non-smoothness of the function did not cause any observable issues. In particular, the styrofoam balls we used to fabricate the results in Fig. 1, came in integer sizes from 1cm to 10cm, 12cm and 14cm in diameter. An illustration of the planar and spherical cases is shown in Fig. 3 and 4.

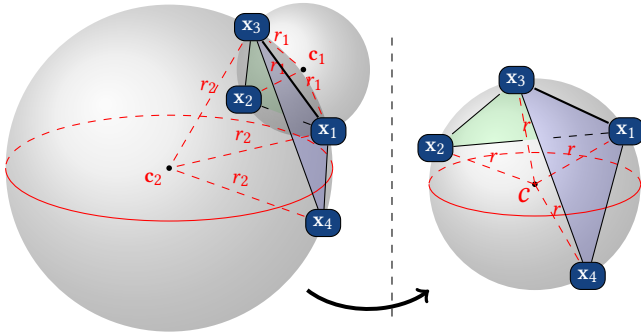
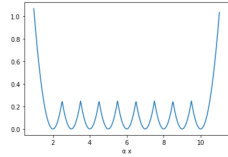


Figure 4: An illustration showing two triangles and their corresponding spheres before and after matching.

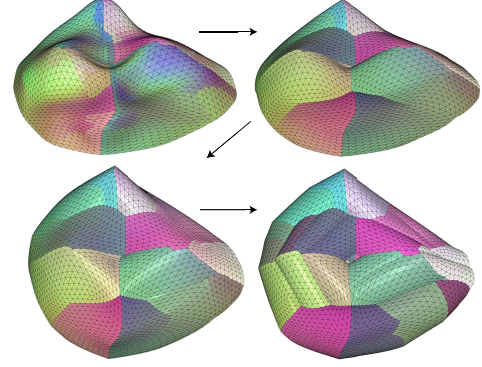


Figure 5: Panelization of a shape with cylindrical panels.

Discussion. Instead of using normals as variables, an alternative would be to express them as a function of the vertices, thus reducing problem size. We experimented with this and, while we obtained similar results, it was not obvious how to extend this formulation to other panel shapes, in particular spheres. We considered using *hinges*, i.e. pairs of triangles sharing an edge to express the centers and radii, and ensure that two adjacent hinges have the same center and radius. We also experimented with one-rings as basic units, but in both cases were unable to achieve clearly defined panels.

We also note that, although not explored in depth, our approach is likely to be mesh dependant. Indeed, while triangles can deform a lot, the specific tessellation used does induce bias. However, we argue that in an interactive setting, the role of mesh independence plays a less significant role. Nevertheless, investigating the influence of the tessellation on the result, or alternatively, formulating a mesh independent problem would be an interesting avenue of research.

3.3 Additional objectives

Regularization. The regularization term ensures that the result does not deviate too much from the original design. We employed standard regularization terms, such as vertex distances (VD) to their original positions, and uniform and cotangent Laplacian smoothing (UL and CL), which are the norm of the difference between a vertex and the weighted average of its neighbors. Setting a high weight to the distance term would result in a shape more faithful to the original one, but with more panels. Conversely, Laplacian smoothing would reduce sharp features, with cotangent weights preserving the shape of the triangles better, if that is deemed necessary. In addition, we used a surface version of the Symmetric Dirichlet (SD) energy [Smith and Schaefer 2015] to encourage less stretching and avoid triangle overlaps. This energy is guaranteed to produce inversion free deformations in the 2D case, but not for surface deformation. Nevertheless, using this energy improves results considerably, as can be seen in Fig. 7. To evaluate this energy for one triangle, we compute the Jacobian of the transformation from the source triangle in its tangent plane, to its deformed shape in 3D, which results in a 3×2 matrix. We then project the Jacobian onto the tangent plane of the target, to achieve a 2×2 Jacobian, for which the expression for the SD energy are readily available. The weights we used for the examples in this paper are $w_c = 1.6$, 0.5 for SD, 0.3 for VD, 0.05 for UL and 0 for CL.

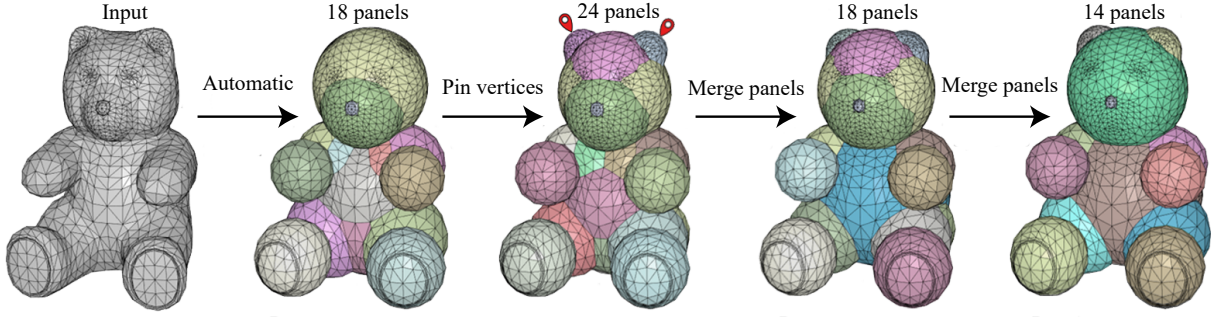


Figure 6: Our interface provides editing tools to improve the automatically computed panelization. These tools include pinning vertices and encouraging the merging and splitting of panels. Please refer to the accompanying video for further demonstrations.

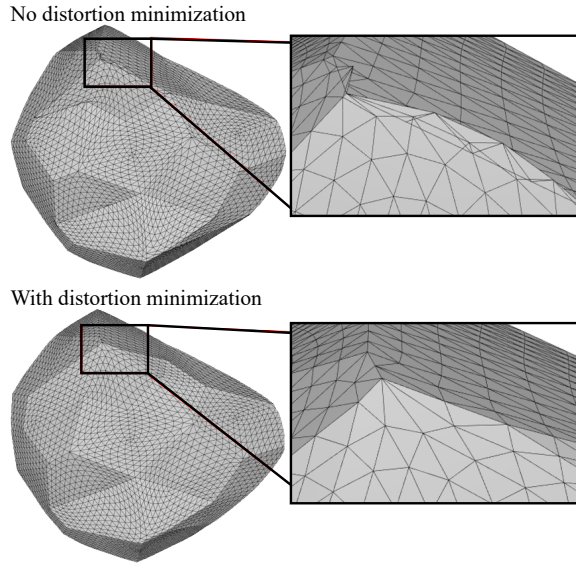


Figure 7: Without distortion minimization, triangles are more prone to overlap.

User defined objectives. We have included additional objectives to assist a user in refining the results. Specifically, we added a simple quadratic objective that allows the user to pin and reposition selected vertices to desired positions. We additionally allow the user to mark regions which should belong to the same panel. This informs another objective that minimizes the squared norm of the difference between $\mathbf{p}_i$ and $\mathbf{p}_j$ of adjacent faces. Conversely, the user can also suggest edges that should form creases between two panels, and split a panel in two. For these, we simply give a zero weight to the corresponding edges in the ℓ_0 norm term in 2. Finally, we allow the user to indicate two panels that should be merged. To do so, we run a simple clustering algorithm [Paredes and Vargas 2012] to detect all faces that would belong to the same panel. We then take all faces belonging to the two clusters, and encourage them to have similar parameters. We note that the clustering is not an integral part of the optimization. An example of an editing process is shown in Fig. 6.

3.4 Optimization

We experimented with different methods for solving (2), including Newton’s method, L-BFGS, and ADAM [Kingma and Ba 2014], but opted to use ADAM for its performance in the interactive setting. The initialization of the parameters for the problem plays an important role in the quality of the results when no user interaction is desired. For the planar case, we simply used the unit normal for each face. For the spherical case we employed two different approaches. One approach is to approximate the parameters for each face by fitting a sphere to the vertices around that face. We found two limitations to this approach. First, it will generate spheres with very large radii in flat regions, and it fails to capture the geometry well in cylindrical regions. Second, it has a greater tendency to lead to spherical panels that are *concave*, with respect to the shape. While this is not a problem if the intended fabrication approach is to use spherical shells, it makes fabrication with solid spheres impossible. The other approach, which worked surprisingly well, was to use the same triangle normals used for the planar case as initialization. For the cylindrical case, the center and radius are chosen similarly to the spherical case, while the direction is chosen at random.

4 RESULTS

We have implemented our method in C++ using libigl [Jacobson et al. 2018], an open source C++ geometry processing library. We ran our experiments on a 8-core intel(R) Core(TM) i7-9700 CPU clocked at 3.0 GHz. The time per iteration depends on the number of objectives used and the complexity of the input model. For a 4K triangle model, one iteration requires around 2-5ms, while for a 20K triangle models, one iteration typically requires about 12-30ms.

We have tested our method in two operation modes on a number of different models. In the automatic mode, the sharpening factor δ is halved after a user-defined but fixed number of iterations. Sharpening more frequently will generally lead to faster convergence, but tends to generate more panels. We note that, in most cases, the result obtained after just a few seconds will already appear very similar to the final result—see also the comparison in Fig. 10. In addition to the automatic approach, we allow the user to manually refine the results using a variety of tools, as discussed in the previous section, and as shown in Fig. 6. We also refer the reader to the accompanying video for more results.

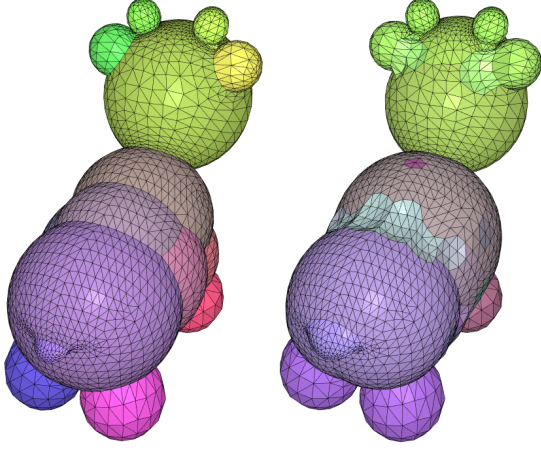


Figure 8: Concave regions can appear for a certain type of initialization, as can be seen on the right. These might be undesirable, depending on the fabrication method.

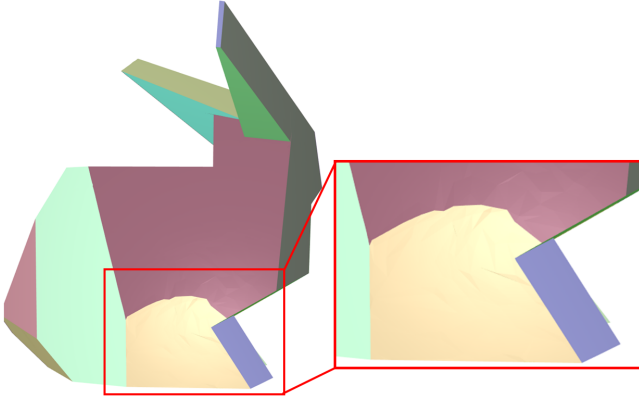


Figure 9: A result obtained with [Chen et al. 2013] contains non planar panels.

For a quantitative evaluation of our results, we measure the deviations of each panel from being a perfectly planar or spherical. We use the norm of the difference between adjacent normals in the planar case, and the distance from the sphere in the spherical case. We compare our results to the method presented in [Chen et al. 2013] for planar panelization, where we used the authors' own implementation. Their approach generally requires many trial-and-error iterations before a valid, competitive result is produced, meaning that a single attempt will require several minutes or even hours. Our best attempt still generated highly non planar panels (See Fig. 9). We provide a summary of the results in Table 1.

5 DISCUSSION

We presented an approach for surface panelization as a continuous deformation process. The proposed method is accompanied by a user interface that allows the optimization to be guided toward the

Table 1: Quantitative results, comprairng our method with no user guidance to [Chen et al. 2013], for various sets of parameters. The competing approach requires considerable experimentation before it is able to produce usable results, and requires between minutes to hours to terminate, with no visual feedback in the meanwhile. For visual results see figure 10.

Mesh Name	# Panels	Max deviation	Avg. deviation	Overlapping faces
Bunny planar [2013]	57	2.0	$6.5 \cdot 10^{-3}$	3
	73	1.9	$5.8 \cdot 10^{-3}$	6
	88	1.7	$3.3 \cdot 10^{-3}$	4
Bunny planar	84	$5.1 \cdot 10^{-3}$	$0.4 \cdot 10^{-3}$	2
	88	$1.9 \cdot 10^{-3}$	$8.0 \cdot 10^{-5}$	1
	92	$1.2 \cdot 10^{-8}$	$1 \cdot 10^{-9}$	0
Koala planar	74	$1.9 \cdot 10^{-4}$	$7.8 \cdot 10^{-6}$	6
	79	$5.3 \cdot 10^{-4}$	$2.4 \cdot 10^{-5}$	2
	89	$9.4 \cdot 10^{-4}$	$1.0 \cdot 10^{-5}$	6
Bear spherical	14	$1.5 \cdot 10^{-4}$	$3.3 \cdot 10^{-7}$	0
	25	$2.5 \cdot 10^{-4}$	$1.2 \cdot 10^{-4}$	2
	30	$4.4 \cdot 10^{-4}$	$1.5 \cdot 10^{-6}$	0
Spot spherical	60	$1.7 \cdot 10^{-4}$	$9.1 \cdot 10^{-7}$	0
	38	$2.2 \cdot 10^{-4}$	$8.3 \cdot 10^{-7}$	0
	25	$1.1 \cdot 10^{-4}$	$3.2 \cdot 10^{-7}$	0

desired target. While the application presented in this paper can stand on its own as a design tool, we believe that the strength of the approach is its formulation. As suggested in the introduction, rationalization is just one of many steps in the design pipeline. The continuous formulation means that the solution can be integrated into a larger optimization problem, thus eliminating one source of uncertainty. As an optimization problem, its derivatives can be obtain through sensitivity analysis, as has been demonstrated in many applications in recent years, in differentiable simulators, renderers, etc. There are many fine details involved in the process, that are outside the scope of this paper, and we leave them to future work.

The inspiration to use different panel shapes comes from real architectural projects, such as the Eden project, the Amazon Spheres, the National Aquatics Centre in Beijing (also known as Watercube), and the SKUM pavilion. Based on our observations and conversations with architects and designers are constantly seeking new ways of materializing structures and facades. Polygonal panels offer ways of distinguishing landmark projects such as these examples. An interactive system further allows designer to explore. However, the final result also highly depends on the route taken by the user, which makes reproducing the same result more difficult. A related challenge that still remains, which might be desirable in some cases, is optimizing more global properties of the panels. For example, due to fabrication or assembly considerations, the dimensions of the panels or their weight might be constrained. In this paper, we considered only objectives expressed in terms of local geometry and deformation properties. Optimizing a panel property like area is challenging in our framework since it requires identifying all of its constituent triangles. Of course, these modifications could be made

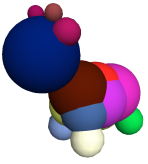
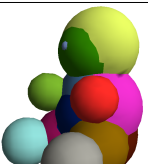
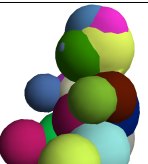
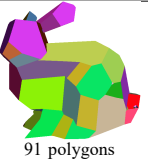
Input Mesh	Output			
				
5,856 faces 2,930 vertices	23 spheres 81 sec	38 spheres 28 sec	60 spheres 12 sec	
				
2,912 faces 1,458 vertices	13 spheres 21 sec	21 spheres 7 sec	30 spheres 5 sec	
				
5,632 faces 2,844 vertices	72 polygons 70 sec	79 polygons 52 sec	88 polygons 40 sec	
				
10k faces 6,164 vertices	83 polygons 5 min	91 polygons 3 min	102 polygons 2 min	

Figure 10: Additional results.

in a separate post-process once the rationalization is completed. However, this would go against the goal of unifying the design pipeline. Our plan for tackling objectives of this type is to compute a *soft* assignment of faces to panels. This soft assignment should allow us to optimize certain properties such as area and panel regularity. However, properties that rely on the combinatorics of the rationalization, e.g. fairness, will likely remain a challenge for future work. Nevertheless, some of these concerns may be resolved using more clever design tools.

ACKNOWLEDGMENTS

We would like to thank Krispin Wandel, Sarah Schneider, Matthias Kohler and Fabio Gramazio for their input. This research was funded in part by GIF Grant no. 2530.

REFERENCES

- Guy Austern, Guedi Capeluto, and Jacob Grobman. 2018. Rationalization methods in computer aided fabrication: A critical review. *Automation in Construction* 90 (06 2018). <https://doi.org/10.1016/j.autcon.2017.12.027>
- Amit H. Bermano, Thomas Funkhouser, and Szymon Rusinkiewicz. 2017. State of the Art in Methods and Representations for Fabrication-Aware Design. *Comput. Graph. Forum* 36, 2 (may 2017), 509–535.
- Bernd Bickel, Paolo Cignoni, Luigi Malomo, and Nico Pietroni. 2018. State of the Art on Stylized Fabrication. *Computer Graphics Forum* 37 (2018). <http://vcg.isti.cnr.it/>

- Publications/2018/BCMP18
- Sofien Bouaziz, Mario Deuss, Yuliy Schwartzburg, Thibaut Weise, and Mark Pauly. 2012. Shape-Up: Shaping Discrete Geometry with Projections. *Computer Graphics Forum* 31, 5 (2012), 1657–1667.
- Christopher Brandt, Hans-Peter Seidel, and Klaus Hildebrandt. 2015. Optimal Spline Approximation via ℓ_0 -Minimization. *Computer Graphics Forum* 34, 2 (2015), 617–626.
- Desai Chen, Pitchaya Sitthi-amorn, Justin T. Lan, and Wojciech Matusik. 2013. Computing and Fabricating Multiplanar Models. *Computer Graphics Forum* 32, 2pt3 (2013), 305–315. arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.12050>
- David Cohen-Steiner, Pierre Alliez, and Mathieu Desbrun. 2004. Variational Shape Approximation. *ACM Trans. Graph.* 23, 3 (Aug. 2004), 905–914.
- Barbara Cutler and Emily Whiting. 2007. Constrained Planar Remeshing for Architecture. In *Proceedings of Graphics Interface 2007* (Montreal, Canada) (*GI '07*). Association for Computing Machinery, New York, NY, USA, 11–18.
- Michael Eigensatz, Martin Kilian, Alexander Schiftner, Niloy J. Mitra, Helmut Pottmann, and Mark Pauly. 2010. Paneling Architectural Freeform Surfaces. *ACM Trans. Graph.* 29, 4, Article 45 (jul 2010), 10 pages.
- Simon Flöry and Helmut Pottmann. 2010. Ruled Surfaces for Rationalization and Design in Architecture. In *LIFE in-formation. On Responsive Information and Variations in Architecture*, Aaron Sprecher, Shai Yeshayahu, and Pablo Lorenzo-Eiroa (Eds.). Association for Computer Aided Design in Architecture (ACADIA), 103–109. Proc. ACADIA 2010.
- Lei He and Scott Schaefer. 2013. Mesh Denoising via L_0 Minimization. *ACM Trans. Graph.* 32, 4, Article 64 (jul 2013), 8 pages.
- Kristian Hildebrand, Bernd Bickel, and Marc Alexa. 2012. crdbd: Shape Fabrication by Sliding Planar Slices. *Computer Graphics Forum* 31, 2pt3 (2012), 583–592. <https://doi.org/10.1111/j.1467-8659.2012.03037.x> arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1467-8659.2012.03037.x>
- Michael Holroyd, Ilya Baran, Jason Lawrence, and Wojciech Matusik. 2011. Computing and Fabricating Multilayer Models. In *Proceedings of the 2011 SIGGRAPH Asia Conference* (Hong Kong, China) (SA '11). Association for Computing Machinery, New York, NY, USA, Article 187, 8 pages. <https://doi.org/10.1145/2024156.2024221>
- Alexandra Ion, Michael Rabinovich, Philipp Herholz, and Olga Sorkine-Hornung. 2020. Shape Approximation by Developable Wrapping. *ACM Transactions on Graphics (Proceedings of SIGGRAPH ASIA)* 39, 6 (2020). <https://doi.org/10.1145/3414685.3417835>
- Alec Jacobson, Daniele Panozzo, et al. 2018. libigl: A simple C++ geometry processing library. <https://libigl.github.io/>.
- Diederik P. Kingma and Jimmy Ba. 2014. Adam: A Method for Stochastic Optimization. cite arxiv:1412.6980Comment: Published as a conference paper at the 3rd International Conference for Learning Representations, San Diego, 2015.
- Yufei Li, Yang Liu, and Wenping Wang. 2015. Planar Hexagonal Meshing for Architecture. *IEEE Transactions on Visualization and Computer Graphics* 21, 1 (2015), 95–106. <https://doi.org/10.1109/TVCG.2014.2322367>
- Hsueh-Ti Derek Liu and Alec Jacobson. 2019. Cubic Stylization. *ACM Trans. Graph.* 38, 6, Article 197 (nov 2019), 10 pages.
- Hsueh-Ti Derek Liu and Alec Jacobson. 2021. Normal-Driven Spherical Shape Analogies. arXiv:2104.11993 [cs.GR]
- Yang Liu, Helmut Pottmann, Johannes Wallner, Yong-Liang Yang, and Wenping Wang. 2006. Geometric Modeling with Conical Meshes and Developable Surfaces. *ACM Trans. Graph.* 25, 3 (jul 2006), 681–689. <https://doi.org/10.1145/1141911.1141941>
- James McCrae, Karan Singh, and Niloy J Mitra. 2011. Slices: a shape-proxy based on planar sections. *ACM Trans. Graph.* 30, 6 (2011), 168.
- Gonzalo E. Paredes and Luis S. Vargas. 2012. Circle-Clustering: A new heuristic partitioning method for the clustering problem. In *The 2012 International Joint Conference on Neural Networks (IJCNN)*. 1–8.
- Davide Pellis, Martin Kilian, Helmut Pottmann, and Mark Pauly. 2021. Computational Design of Weingarten Surfaces. *ACM Trans. Graph.* 40, 4, Article 114 (jul 2021), 11 pages.
- Kacper Pluta, Michal Edelstein, Amir Vaxman, and Mirela Ben-Chen. 2021. PH-CPF: Planar Hexagonal Meshing Using Coordinate Power Fields. *ACM Trans. Graph.* 40, 4, Article 156 (jul 2021), 19 pages.
- Roi Poranne, Marco Tarini, Sandro Huber, Daniele Panozzo, and Olga Sorkine-Hornung. 2017. Autocuts: Simultaneous Distortion and Cut Optimization for UV Mapping. *ACM Trans. Graph.* 36, 6, Article 215 (Nov. 2017), 11 pages.
- Helmut Pottmann, Michael Eigensatz, Amir Vaxman, and Johannes Wallner. 2015. Architectural geometry. *Computers & Graphics* 47 (2015), 145–164.
- Silvia Sellán, Noam Aigerman, and Alec Jacobson. 2020. Developability of Heightfields via Rank Minimization. *ACM Transactions on Graphics* (2020).
- Mayank Singh and Scott Schaefer. 2010. Triangle Surfaces with Discrete Equivalence Classes. In *ACM SIGGRAPH 2010 Papers* (Los Angeles, California) (*SIGGRAPH '10*). Association for Computing Machinery, New York, NY, USA, Article 46, 7 pages.
- Jason Smith and Scott Schaefer. 2015. Bijective Parameterization with Free Boundaries. *ACM Trans. Graph.* 34, 4, Article 70 (July 2015), 9 pages.
- Oded Stein, Eitan Grinspun, and Keenan Crane. 2018. Developability of Triangle Meshes. *ACM Trans. Graph.* 37, 4 (2018), 30.

- Chengcheng Tang, Xiang Sun, Alexandra Gomes, Johannes Wallner, and Helmut Pottmann. 2014. Form-finding with polyhedral meshes made simple. *ACM Transactions on Graphics (TOG)* 33, 4 (2014), 1–9.
- Charlie C.L. Wang and Gershon Elber. 2014. Multi-dimensional dynamic programming in ruled surface fitting. *Computer-Aided Design* 51 (2014), 39 – 49.
- Henrik Zimmer, Marcel Campen, David Bommes, and Leif Kobbelt. 2012a. Rationalization of Triangle-Based Point-Folding Structures. *Computer Graphics Forum* 31, 2pt3 (2012), 611–620. arXiv:<https://onlinelibrary.wiley.com/doi/pdf/10.1111/j.1467-8659.2012.03040.x>
- Henrik Zimmer, Marcel Campen, Ralf Herkrath, and Leif P. Kobbelt. 2012b. Variational Tangent Plane Intersection for Planar Polygonal Meshing. In *AAG*.